A PROJECT REPORT ON

Image Caption Generator

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Sagar Naphade	Exam No: B1900504413
Prathamesh Landge	Exam No: B1900504365
Ayush Tolani	Exam No: B1900504222

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025

CERTIFICATE

This is to certify that the project report entitled

Image Caption Generator

Submitted by

Sagar Naphade	Exam No: B1900504413
Prathamesh landge	Exam No: B1900504365
Ayush Tolani	Exam No: B1900504222

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. V. S. Gaikwad
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date:

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'Image Caption Generator'****.*

We would like to thank our project guide ***Prof. V. S. Gaikwad*** *for giving us all the help and guidance we needed. We are really grateful to them for their kind support and valuable suggestions.*

We are also grateful to ***Dr. G. V. Kale****, our project co-guide and Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to* ***Dr. A. R. Deshpande*** *BE project coordinator, for her timely guidance and support.*
.

Our special thanks to our honourable ***Dr. P. T. Kulkarni****, Director,PICT and* ***Dr. S. T. Gandhe****, Principle, PICT for their continued support and valuable inputs during ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Sagar Naphade
Prathamesh landge
Ayush Tolani
(B.E. Computer Engg.)

ABSTRACT

The integration of computer vision and natural language processing has resulted in considerable failures in allowing machines to interpret and communicate visual content. One of the most compelling consequences of this integration is the development of caption systems that automatically generate descriptive sentences for a particular image. This project focuses on generating captions using deep learning-based architectures to effectively accomplish this task. In particular, the VGG16 model is used to extract characteristics from images with high levels of semantic and spatial details. These extracted properties are handed over to a BILSTM network (bidirectional long-term memory) that handles the continuous nature of the language and creates contextual capabilities. Using bidirectional structures allows the model to consider both past and future contexts in the caption, leading to a more accurate and flowing explanation. The system is trained and evaluated on yield data records using standard images, and its performance is assessed using metrics such as BLEU, METEOR, and ROUGE-L. A comparative analysis is also performed based on these metrics. The results highlight the promise to combine visual perception with natural language in order to create intelligent systems that allow for a comprehensive understanding of the scene.

Contents

List of Figures

CHAPTER 1
INTRODUCTION

1.1 Overview

This project focuses on developing automated capacity systems, allowing descriptive text to be generated for images. The system combines the features of a folding folding network (CNNS) for image function extraction, and a repeating neural network (RNN) for language generation. Trained with Imagenet data records, VGG16 is used without a final classification layer to output a compact, meaningful presentation of the input form. This functional vector serves as input to the speech model. In contrast to traditional LSTMs, Bi-LSTM processes sequences both forward and backward, so the model records context more effectively and improves the quality of generated captions. This model learns to combine visual features with corresponding textual descriptions. Adam Optimizer is used to train models efficiently by adapting learning rates during training. After training, the model can generate grammar and context-related captions for images.

1.2 Motivation

In today's digital age, the amount of shared visual data generated is increasing exponentially. From social media platforms to medical imaging and surveillance materials, the need to automatically understand and process images is more important than ever. People can easily interpret and explain visual scenes, but the possibility of doing the same remains an important challenge in artificial intelligence. By combining the power of computer vision and deep learning in natural language processing, we want to create a system that can automatically generate human descriptions of images. E-commerce: Captioning product images, SEO improvements, user improvements. It serves as a practical application and as a valuable learning experience in building AI systems that can "see" and "speak."

1.3 Problem Definition and Objectives

Humans possess an innate ability to effortlessly grasp and articulate intricate details about their surroundings. With just a cursory glance at an image, humans can provide rich and insightful descriptions. This cognitive skill is fundamental to our species. For years, researchers in the realm of artificial intelligence have been striving to replicate this exceptional human capability in computers. Although great progress has been made in various computer vision tasks, such as object recognition, attribute classification, action classification

, image classification and scene recognition , it is a relatively new task to let a computer use a human-like sentence to automatically describe an image that is forwarded to it.

Image captioning serves as an integration point for computer vision and natural language processing, requiring a seamless collaboration between the two fields. The field of computer vision has opened up a world of possibilities in various applications due to its ability to enable computers to describe the visual world. This advancement has paved the way for natural human robot interactions, early childhood education, information retrieval, visually impaired assistance.

As technology continues to evolve, automatic image captioning techniques are expected to become even more sophisticated and accurate. By leveraging advanced approaches such as concept hierarchies and language templates, researchers are making significant strides in the field of image captioning, paving the way for a future where detailed and comprehensive image descriptions are generated effortlessly.

1.4 Project Scope & Limitations

The scope of this project includes image preprocessing, feature extraction using the pre-trained VGG16 Convolutional Neural Network (CNN), and sentence generation using a Bidirectional Long Short-Term Memory (Bi-LSTM) network. The system is trained using a dataset of image-caption pairs and optimized using the Adam optimizer to enhance performance and learning efficiency. The project also includes evaluation using standard NLP metrics such as BLEU scores to assess caption quality. The final system is capable of generating descriptive sentences for previously unseen images, with potential applications in fields such as accessibility for visually impaired users, automated content generation, image search optimization, and digital media management.

While the Image Caption Generator demonstrates promising results, it has several limitations. The model's performance heavily depends on the quality, size, and diversity of the training dataset; it may struggle with images that depict abstract scenes or objects not present in the dataset. The vocabulary is limited to the words seen during training, which can result in repetitive or generic captions. Moreover, training deep learning models like this one requires significant computational resources, which may be a constraint in certain development environments.

1.5 Methodologies of Problem solving

The development of the Image Caption Generator involves a combination of computer vision and natural language processing techniques. The methodology follows a systematic pipeline designed to transform raw images into meaningful text descriptions.

The process begins by collecting a dataset consisting of images and their corresponding captions. Commonly used datasets for this purpose include Flickr8k, Flickr30k, or MS COCO, which provide multiple human-annotated captions per image.

Preprocessing involves Resizing images to fit the input requirements of the VGG16 model.Cleaning and tokenizing the captions (removing punctuation, converting to lowercase, etc.).Creating a vocabulary of unique words used in the captions.Padding sequences to ensure uniform input length.

VGG16 Convolutional Neural Network is used for extracting high-level features from images. The top classification layer is removed, and the output of the last convolutional layer is used to obtain a fixed-length feature vector for each image. This reduces the complexity of the image and converts it into a format that can be combined with text.

To generate captions, the project uses a Bidirectional Long Short-Term Memory (Bi-LSTM) network. Unlike unidirectional LSTMs, Bi-LSTM processes input sequences from both directions, helping the model to better understand the context and structure of a sentence. The feature vector from the CNN is combined with the word embeddings of the caption and fed into the Bi-LSTM to predict the next word in the sequence.

The model is trained on image-caption pairs using supervised learning. During training, the system learns to associate image features with sequences of words. The Adam optimizer is employed to improve training speed and accuracy by adapting the learning rate based on the gradient behavior. A categorical cross-entropy loss function is used to evaluate prediction accuracy during training.

Once trained, the model can generate captions for new, unseen images. The process begins with a start token, and the model predicts one word at a time, appending each predicted word to the input sequence until an end token is generated or the maximum length is reached. The performance of the caption generator is evaluated using metrics like the BLEU (Bilingual Evaluation Understudy) score, which compares the generated captions with reference human-written captions based on n-gram overlap. Higher BLEU scores indicate better performance.

CHAPTER 2
LITERATURE SURVEY

2.1 Literature Survey

The task of generating natural language captions for images is an interdisciplinary challenge that involves both computer vision and natural language processing (NLP). Over the past decade, significant progress has been made in this field, with a shift towards deep learning models that combine convolutional neural networks (CNNs) for image feature extraction and recurrent neural networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, for language generation.

Early attempts at image captioning relied heavily on traditional computer vision techniques such as object detection and image segmentation. These methods manually identified objects or scenes in the image and used predefined templates to generate captions. For instance, Feng et al. [11] developed a system that used probabilistic graphical models to learn the relationships between objects and actions in images. However, these systems were limited in their ability to generate creative, human-like captions, as they relied on predefined rules and lacked context understanding.

The introduction of deep learning revolutionized image captioning. In 2014, the seminal work by Vinyals et al. [12] titled *Show and Tell: A Neural Image Caption Generator* marked a major breakthrough in this field. They proposed an end-to-end neural network model that combined a CNN (specifically, a GoogleNet architecture) for image feature extraction and an RNN for sequence modeling of captions. This architecture could generate captions directly from raw image pixels, with no manual feature engineering required. Their model was one of the first to achieve promising results in generating coherent captions that were contextually relevant to the image.

In a similar vein, Karpathy et al. [13] introduced a more refined approach by combining the VGG16 CNN for image feature extraction with an LSTM network for language generation. This model, known as the CNN-LSTM architecture, became widely used in subsequent image captioning models. The LSTM network allowed the model to capture long-range dependencies in language, improving the fluency and accuracy of the generated captions.

One significant limitation of CNN-LSTM models is that they treat the image as a whole, ignoring the spatial relationships between objects within the image. This problem was addressed by Xu et al. [14] in their paper *Show, Attend and Tell: Neural Image Caption Generation with Visual Attention*. They introduced an attention mechanism, which allowed the model to focus on specific regions of the image when generating captions. This method improved caption quality by enabling the model to generate more detailed and contextually relevant descriptions of images.

In attention-based models, the network learns to assign different weights to various regions of the image based on the words it is generating, thus making the caption generation process more precise. This advancement has led to better captioning, particularly for images with multiple objects or complex scenes.

The advent of transformer-based architectures has further enhanced the performance of image captioning systems. The Transformers model, introduced by Vaswani et al. [15] in their work *Attention Is All You Need*, has since become the backbone of many NLP applications due to its ability to handle long-range dependencies more efficiently than LSTMs. Li et al. [16] proposed using transformers for image captioning, where they combined vision transformer models (ViT) for feature extraction and transformers for language modeling. Their model showed superior performance over traditional CNN-LSTM and attention-based models, particularly in generating captions that are more fluent and contextually accurate.

ImageBERT by Chen et al. [17] is another transformer-based model that utilizes both image and text data simultaneously, leveraging a shared transformer architecture to jointly learn image-text embeddings. This model outperforms CNN-based architectures in terms of caption relevance and accuracy.

Evaluation of image captioning systems has always been a challenging aspect of research, as traditional machine learning metrics like accuracy or loss are not well-suited for assessing the quality of text. Several automated metrics have been proposed to measure the quality of generated captions, including:

- **BLEU Score** by Papineni et al. [18]: A metric that evaluates the overlap of n-grams between the generated caption and reference captions.
- **METEOR** by Banerjee et al. [19]: A metric that combines precision, recall, and synonym matching to evaluate caption quality.
- **ROUGE** by Lin et al. [20]: A metric primarily used for evaluating recall and the quality of generated text compared to reference captions.
- **CIDEr** by Vedantam et al. [21]: A metric designed to evaluate the consensus between multiple human-generated captions and the model's output.

These metrics have become standard tools for evaluating the performance of image captioning models.

Recent research in image captioning has increasingly focused on multimodal learning, where models learn from both visual and textual data to improve the quality of generated captions. For instance, CLIP (Contrastive Language-Image Pretraining) by Radford et al. [22] uses contrastive learning to align images with textual descriptions. This allows the model to generate captions that are semantically meaningful by learning from a large corpus of image-text pairs.

Additionally, the integration of reinforcement learning has gained attention in improving caption generation by optimizing for human-centric metrics like coherence and informativeness, rather than just n-gram overlap.

The image captioning field has evolved from rule-based systems to sophisticated deep learning models that combine CNNs and RNNs, with significant improvements through attention mechanisms and transformer-based architectures. Although current models have achieved substantial success, challenges such as improving contextual understanding, handling complex scenes, and enhancing real-time performance remain areas for further research. As the field advances, future models may move toward integrating vision-language pretraining and more sophisticated evaluation methods to generate more accurate, creative, and context-aware captions.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

1. **Assumptions:**

   - It is assumed that pre-trained VGG16, InceptionV3, or similar CNN models are available for extracting image features.
   - It is assumed that ethical considerations will be taken into account during model development. The captions generated by the model must avoid biased, offensive, or inappropriate language, ensuring fairness and respect.
   - It is assumed that GPU resources are available to accelerate model training. Training deep learning models requires significant computational power, and having access to a GPU will help speed up the process considerably.
   - Images provided to the system are assumed to be clear, properly lit, and relevant for caption generation.
   - Users (especially end users) will rely on a stable internet connection to upload images, receive captions, and interact with the system.
   - Input images are assumed to be resized and preprocessed to match the expected dimensions of the CNN model (e.g., 224x224 for VGG16).
   - Users will understand that captions are generated automatically and may not be perfect.
   - Users will provide images in compatible formats (e.g., JPG, PNG).

2. **Dependencies:**

   - Users (especially end users) will rely on a stable internet connection to upload images, receive captions, and interact with the system.
   - Users (especially those working on model training) depend on access to GPU resources or high-performance computing systems. Without these resources, model training will be slow especially when dealing with large datasets and complex architectures.
   - The project depends on access to modern deep learning libraries, such as TensorFlow, Keras, or PyTorch, for building and training the neural network models, including the CNN for feature extraction and the LSTM for caption generation.

- The performance of the image captioning system depends on the ability to use evaluation metrics such as BLEU to assess the quality of generated captions.
- If used online,End users depend on the system adhering to appropriate security measures and privacy regulations to ensure that their data (especially images and captions) is handled securely. They expect the platform to protect their information and avoid misuse.

3.2 Functional Requirements

3.2.1 Image Upload Functionality

The system must allow users to upload images(.jpg , .png) via a user interface (e.g., web app) or API. This is the starting point of the captioning process. Without an image upload option, the system cannot generate captions.

3.2.2 Image Preprocessing

The system shall preprocess uploaded images by resizing, normalizing, and formatting them as required by the CNN encoder (e.g., VGG16 expects 224x224 input).

3.2.3 Feature Extraction using CNN)

A pre-trained convolutional neural network (like VGG16) is used to extract visual features from the image and converts raw image data into a high-level representation that the LSTM can understand.

3.2.4 Text Tokenization and Vocabulary Management

Captions in the dataset are tokenized, and a vocabulary dictionary is built mapping each word to an index and required for both training and inference. Tokenized words are converted into sequences that the LSTM can process.

3.2.5 Caption Generation using Bi-directional LSTM

The extracted image features are fed into a Bi-directional LSTM, which generates a sequence of words (a caption).It translates visual features into human-readable language, capturing both forward and backward context.

3.2.6 Caption Decoding

The system shall convert the predicted word tokens back into human-readable sentences using the trained tokenizer.

3.2.7 Model Training

The system shall support training the captioning model using labeled datasets.

3.2.8 Model Evaluation

The system shall evaluate performance using metrics such as BLEU, METEOR, and CIDEr.

3.2.9 Real-time Caption Generation

The system shall generate captions in real-time or near real-time after image upload.

3.3 External Interface Requirements

3.3.1 User Interfaces

- The system shall provide a graphical user interface (GUI) for:
  - Uploading images.
  - Viewing generated captions.
- The GUI will be responsive and accessible on desktops and laptops.

3.3.2 Hardware Interfaces

- The system may require GPU/TPU support for efficient model training and real-time caption generation.
- User-side hardware requirements include:
  - Desktop or laptop with internet access.
- Admin-side hardware should include:
  - Minimum 8GB RAM, multi-core CPU.
  - Optional GPU for high-performance training and evaluation tasks.

3.3.3 Software Interfaces

- **Operating System**: Compatible with Windows, macOS, or Linux.
- **Programming Language**: Python 3.x.
- **Libraries/Frameworks**:
  - TensorFlow/Keras (for model development).
  - NumPy, Pandas, (for data handling and visualization).
  - Streamlit (for web interface or API development).

3.3.4 Communication Interfaces

- **File Formats**:
  - Supported image formats: .jpg, .jpeg, .png.
  - Captions returned in plain text.
- **Input Validation**:
  - File size limit (e.g., 200MB).
  - File type checking before processing.

3.4 System Requirements

3.4.1 Software Requirements (Platform Choice)

Category	**Tools / Technologies**
Deep Learning Framework	TensorFlow 2.x, Keras (integrated in TensorFlow)
Tokenization & NLP Tools	Keras Tokenizer, NLTK (BLEU score evaluation)
Image Processing	VGG16 (for feature extraction)
Interface Development	Streamlit (for web-based prototype/demo)
Supporting Libraries	NumPy, Pandas, Matplotlib, tqdm, pickle
Browser Compatibility	Google Chrome, Safari, Firefox (for frontend)

Table 3.1: Software Requirements for Image Caption Generator System

3.4.2 Hardware Requirements

- Minimum 8GB Desktop/laptop. .

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The architecture of the image captioning system is organized in multiple layers, each responsible for a specific part of the pipeline. The following outlines each layer and its components:

1. Data Collection & Preprocessing Layer

This is the foundational layer where raw data is prepared for model training.

- **Image Dataset** – Collection of images to be captioned.
- **Text Dataset** – Corresponding captions for each image.
- **Preprocess Images** – Resizing, normalization, and other cleaning operations.
- **Tokenization** – Splitting captions into tokens (words/subwords).
- **Vocabulary Creation** – Mapping unique tokens to a vocabulary index.

2. Feature Extraction Layer

- **Load VGG16 Model** – Pre-trained CNN used for feature extraction from images.
- **Extract Feature Vectors** – Output high-level feature representation for each image. This transforms image data into vector format suitable for the model.

3. Text Data Processing Layer

- **Tokenize Captions** – Break captions into tokens.
- **Assign Integer Values** – Map each token to a numeric index.
- **Pad Captions** – Add padding to make all sequences the same length.

4. Model Architecture Layer

This layer integrates both image and text data into a unified deep learning model.

- **Image Feature Input** – Accepts feature vector from CNN.
- **Caption Input** – Accepts processed text input.
- **Input Layer** – Combines both image and caption inputs.
- **LSTM Layer** – Captures temporal dependencies in captions.
- **Decoder Layer** – Generates output word by word.
- **Softmax Activation** – Predicts next word by probability distribution.

5. Training Setup Layer

- **Loss Function** – Computes prediction error (e.g., categorical cross-entropy).
- **Optimizer** – Updates model weights to reduce loss (e.g., Adam).

6. Model Training Layer

- **Training Loop** – Iterative process across epochs.
- **Input Features and Captions** – Paired input to the model.
- **Predicted Captions** – Model-generated text output.
- **Calculate Loss** – Measures how far predictions are from actual captions.
- **Update Weights** – Backpropagation to optimize model parameters.

7. Evaluation Layer

- **Metrics** – Evaluates model performance using metrics like BLEU, METEOR.

4.2 Mathematical Model

We propose a neural and probabilistic framework for generating descriptions from images, inspired by recent advances in statistical machine translation (SMT). In SMT, it is possible to directly maximize the probability of the correct output sequence (caption) given the input (image) using an end-to-end approach. This framework utilizes recurrent neural networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, which are well-suited for handling sequential data like natural language.

Given an image I, our objective is to generate the most likely sentence $\hat{S}$ that describes it. This is formulated as:

$$\hat{S} = \arg\max_{\theta} \sum_{(I,S)} \log p(S|I;\theta) \tag{4.1}$$

Here:

- I is the input image.
- S is the ground truth caption.
- θ denotes the parameters of the model (including weights of CNN and RNN components).
- $\hat{S}$ is the predicted caption.

A. Encoding Phase

The input image I is passed through a convolutional neural network (CNN), such as VGG16 , to extract high-level features. These features are represented as a fixed-length vector:

$$f_I = \text{CNN}(I) \tag{4.2}$$

where $f_I \in \mathbb{R}^d$, and d is the dimension of the extracted feature vector.

B. Decoding Phase

The feature vector f_I is passed to a recurrent neural network (typically an LSTM), which decodes it into a natural language sentence. At each time step t, the model generates a probability distribution over the vocabulary:

$$p(w_t|w_1, w_2, \ldots, w_{t-1}, f_I) = \text{Softmax}(h_t W_o + b_o) \tag{4.3}$$

where:

- w_t is the word generated at time step t,
- h_t is the RNN hidden state at time t,
- W_o and b_o are output weights and bias,
- Softmax outputs the word probability distribution.

C. Training the Model

The model is trained using maximum likelihood estimation (MLE). For a dataset with N image-caption pairs (I_i, S_i), the loss function is:

$$L(\theta) = -\sum_{i=1}^{N} \log p(S_i|I_i;\theta) \tag{4.4}$$

The model parameters θ are optimized using gradient descent methods such as Adam or SGD.

D. LSTM-based Sentence Generator

To mitigate vanishing and exploding gradients, the model employs an LSTM. The LSTM maintains a memory cell c_t, regulated by input, forget, and output gates:

$$i_t = \sigma(W_{ix}x_t + W_{im}m_{t-1}) \quad \text{(Input Gate)} \tag{4.5}$$
$$f_t = \sigma(W_{fx}x_t + W_{fm}m_{t-1}) \quad \text{(Forget Gate)} \tag{4.6}$$
$$o_t = \sigma(W_{ox}x_t + W_{om}m_{t-1}) \quad \text{(Output Gate)} \tag{4.7}$$
$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tanh(W_{cx}x_t + W_{cm}m_{t-1}) \quad \text{(Cell Update)} \tag{4.8}$$
$$m_t = o_t \cdot c_t \quad \text{(Memory Output)} \tag{4.9}$$
$$p_{t+1} = \text{Softmax}(m_t) \quad \text{(Word Prediction)} \tag{4.10}$$

Here, σ is the sigmoid function and tanh is the hyperbolic tangent function. m_t is the output of the memory cell used to predict the next word.

E. Training the LSTM Model

The LSTM is trained to predict each word of the caption conditioned on the image and all previous words. The training objective is:

$$L(\theta) = -\sum_{t=1}^{T} \log p(S_t|I, S_0, \ldots, S_{t-1};\theta) \tag{4.11}$$

where:

- T is the length of the caption,
- S_t is the ground truth word at time t,
- θ represents the model parameters.

The loss is minimized using gradient-based optimization algorithms.

4.3 Data Flow Diagrams

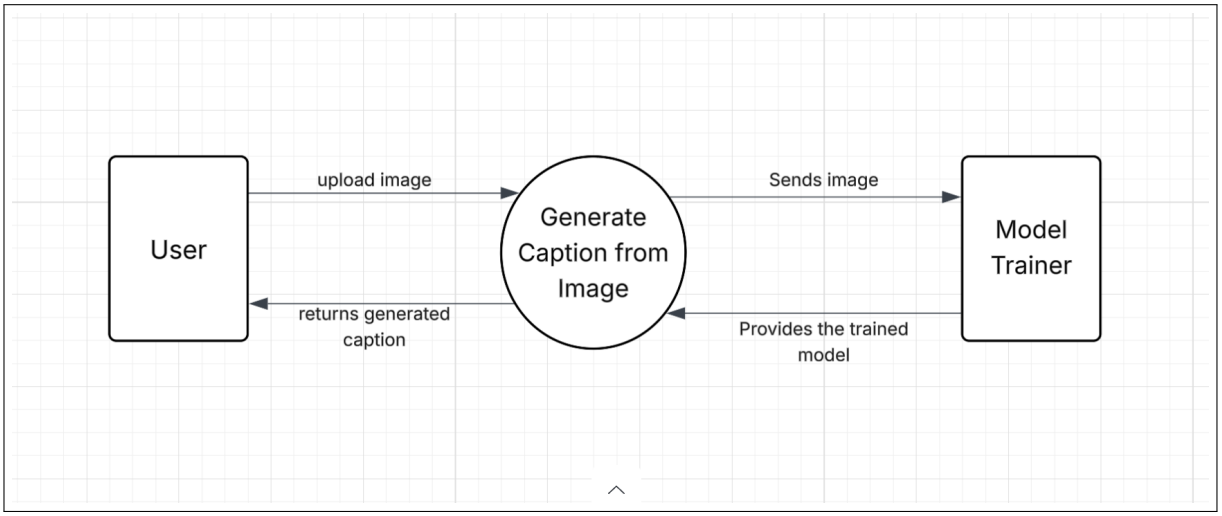

Figure 4.1: Level 0 Data Flow Diagram

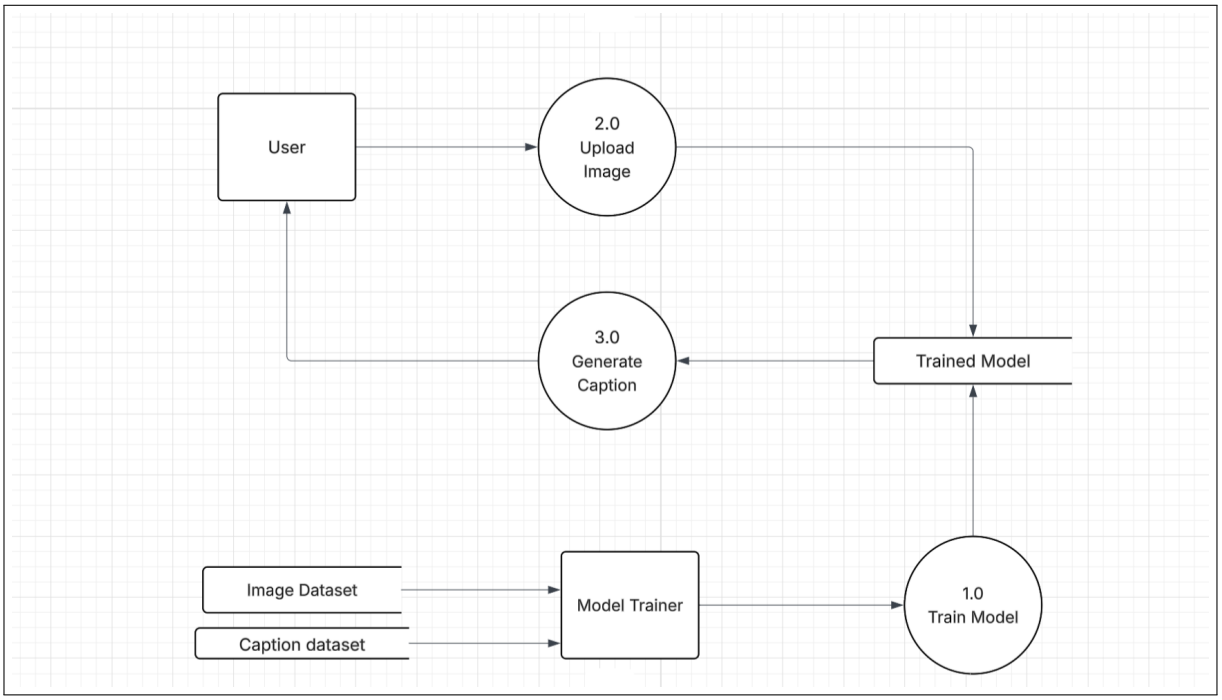

Figure 4.2: Level 1 Data Flow Diagram

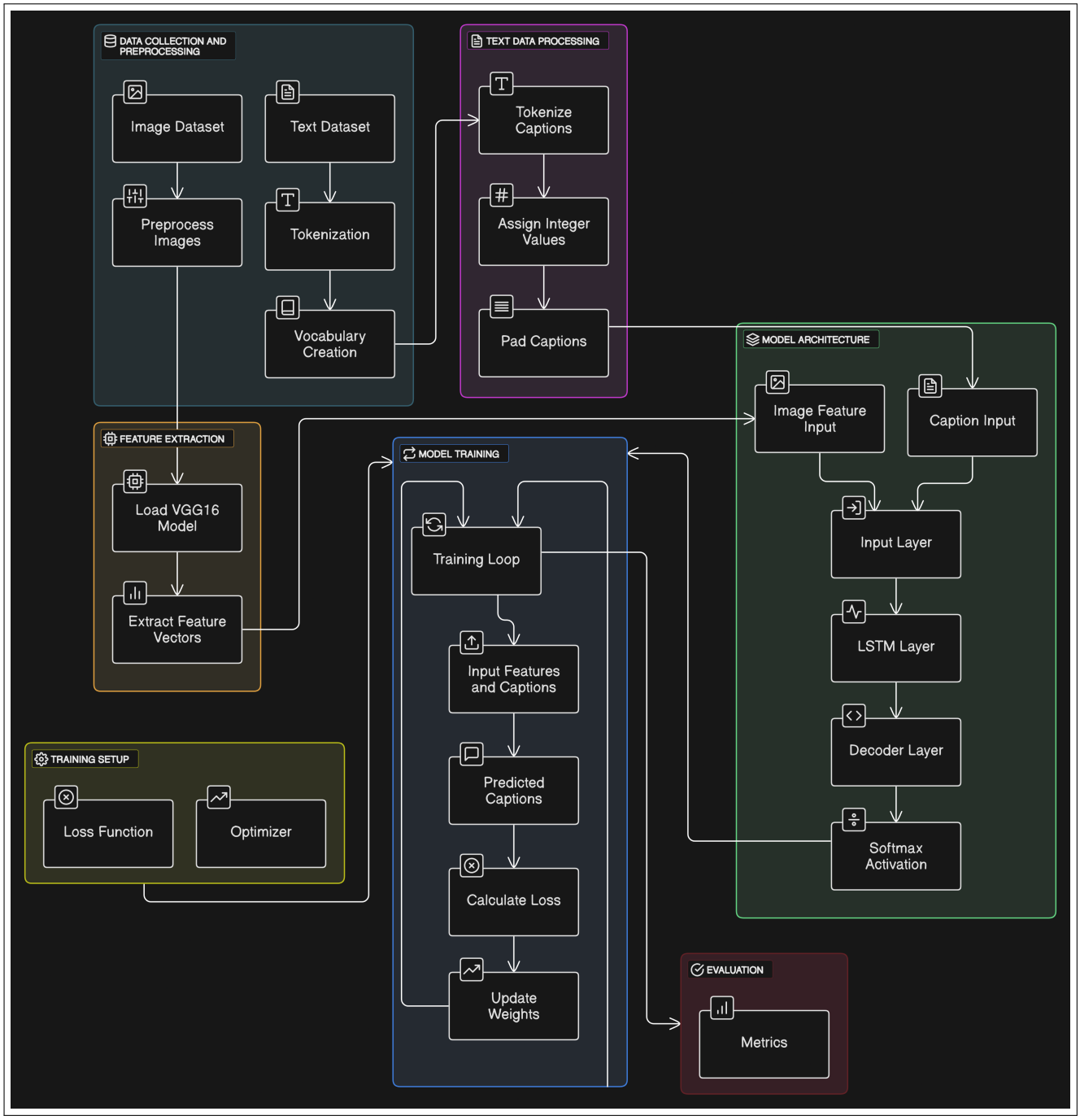

Figure 4.3: System Flow Diagram

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Research and Development:**
  Estimated a total of **5–6 months** for developing and refining the core functionalities of the Image Caption Generator system. This includes researching existing captioning models, preparing datasets (such as MS-COCO or Flickr8k), extracting features using CNNs (e.g., VGG16 or MobileNetV2), and building the caption generation model using Bidirectional LSTMs. It also covers the time required to explore tokenization, embedding layers, and hyperparameter tuning.

- **Testing and Evaluation:**
  Estimated **1.5–2 months** for testing various aspects of the system. This includes model performance evaluation using BLEU scores, user testing of the Streamlit or frontend interface, latency measurement, and accuracy checks across multiple image types. It also involves testing against edge cases like low-resolution or ambiguous images.

- **Documentation and Review:**
  Estimated **1–1.5 months** to create detailed documentation, including system architecture, user guides, API references, and model design explanations. Frequent review sessions will be conducted with project guides and mentors to validate progress, collect feedback, and ensure alignment with academic and practical goals.

5.1.2 Project Resources

To ensure the successful execution of the Image Caption Generator, the following resources are identified:

1. Hardware Resources

- GPU-enabled system
- Personal laptop or PC for coding and testing
- Stable internet connection

2. Software Resources

- **Programming Language:** Python 3.x
- **Libraries and Frameworks:**
  - TensorFlow / Keras
  - NumPy, Pandas
  - Matplotlib
  - NLTK (BLEU scoring)
  - Streamlit
- **Deep Learning Models:** VGG16(CNN), Bidirectional LSTM (RNN)
- **Development Environment:** Jupyter Notebook, VS Code, Streamlit

3. Human Resources

- 3 Developers
- 1 Project Guide
- 1 Tester

4. Data Resources

- MS-COCO dataset
- Flickr8k datasets
- Pre-trained models from Keras (e.g., VGG16)

5. Other Resources

- Research papers
- online articles

5.2 Project Schedule

5.2.1 Project Task Set

- Identify project objectives and scope.
- Define technical stack (VGG16, Bidirectional LSTM, TensorFlow, Streamlit, etc.).
- Document functional and non-functional requirements.
- Acquire dataset (e.g., MS-COCO or Flickr8k/30k).
- Perform image preprocessing (resize, normalize using `preprocess_input`).
- Clean and tokenize captions using Keras Tokenizer.
- Create vocabulary and word-to-index mappings.
- Pad caption sequences for uniformity.
- Load VGG16 model (excluding top layers).
- Extract features from images using CNN.
- Store extracted features in a serialized format (e.g., using `pickle`).
- Define captioning model architecture: CNN + Embedding + Bidirectional LSTM.
- Merge image and text features.
- Compile model using the Adam optimizer.
- Train the model on the dataset and validate with a held-out set.
- Evaluate model performance using BLEU score and confusion matrix.
- Generate sample captions for qualitative evaluation.
- Build a user interface using Streamlit.
- Allow users to upload images and display generated captions.
- Optimize model size and latency for faster inference.
- Create a detailed project report with methodology, architecture diagrams, results, and future scope.

5.2.2 Timeline Chart

August 2024
Finalize problem statement, explore image captioning literature, define scope.

September 2024
Collect datasets (MSCOCO, Flickr8k), define technical requirements, and plan pipeline.

October 2024
Model design: Integrate VGG16 for feature extraction and Bi-LSTM for language generation.

November 2024
Implement image preprocessing, text cleaning, tokenizer setup, and data pipelines.

December 2024
Train the model, tune hyperparameters, validate on BLEU scores, and adjust architecture.

January 2025
Build the user interface using Streamlit. Integrate image upload and prediction display.

February 2025
Perform unit and system testing, verify caption quality, and optimize latency.

March 2025
Package app using Docker, deploy to Microsoft Azure. Prepare documentation.

April 2025
Conduct demo with test users, gather feedback, finalize project report and presentation.

Figure 5.1: Summarized Timeline Chart – Image Caption Generator

5.3 Team Organization

5.3.1 Team structure

- **Ayush Tolani**: Deep Learning Expert, Model Integration Specialist, Team Leader.
- **Sagar Naphade**: Data Preprocessing Expert, Documentation Lead.

- **Prathamesh landge**: Backend Developer, Streamlit App .
- **Prof. V.S. Gaikwad**: Internal Mentor and Project Guide.

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The Image Caption Generator is divided into the following interconnected modules:

1. **Image Preprocessing Module**
   Loads and resizes images, converting them into array format compatible with deep learning models. Uses pre-trained model preprocessing functions.

2. **Feature Extraction Module**
   Employs a CNN (VGG16) to extract feature vectors from input images.

3. **Text Preprocessing Module**
   Tokenizes and cleans caption data, converting it into sequences and applying padding for uniform input length.

4. **Caption Generation Model**
   A deep learning model with embedding, Bidirectional LSTM layers, and dense output for generating captions.

5. **Training and Evaluation Module**
   Trains the model and evaluates its performance using BLEU score and other metrics.

6. **Prediction Module**
   Accepts a new image and generates captions based on trained model inference.

7. **User Interface Module**
   Built with Streamlit to allow users to upload images and generate captions interactively.

8. **Model Persistence Module**
   Handles saving and loading of the trained model and tokenizer for deployment and reuse.

6.2 Tools and Technologies Used

The development of the Image Caption Generator project involved the use of various tools and technologies to facilitate the implementation, training, and deployment of the system. The following list provides an overview of the key technologies used, categorized according to their purpose in the project:

- **Programming Language**
  - **Python**: The entire project is implemented in Python due to its simplicity and the availability of a vast number of libraries for machine learning, deep learning, data processing, and visualization.
- **Deep Learning Frameworks**
  - **TensorFlow 2.x**: An end-to-end open-source platform for machine learning used to build and train the image captioning model.
  - **Keras**: A high-level neural networks API, integrated with TensorFlow, which simplifies the process of building and training deep learning models.
- **Pre-trained CNN Models**
  - **VGG16**: Used for extracting high-level visual features from input images. These models are pretrained on the ImageNet dataset and help in reducing the training time and improving performance.
- **Data Processing and Utility Libraries**
  - **NumPy and Pandas**: Used for numerical operations and data manipulation, such as handling image arrays and caption datasets.
  - **Matplotlib and Seaborn**: Used for data visualization, including loss curves, BLEU scores, and sample outputs.
  - **TQDM**: Provides smart progress bars to monitor loops and training progress.
  - **Pickle**: Used for serializing Python objects such as tokenizers and model configurations.
  - **NLTK**: Natural Language Toolkit, used for evaluating the quality of generated captions using BLEU score.
- **Frontend Development**
  - **Streamlit**: A lightweight Python framework used to develop the web interface that allows users to upload images and display corresponding captions in real-time.
- **Model Evaluation**
  - **BLEU Score (via NLTK)**: The BiLingual Evaluation Understudy Score is used to evaluate the quality of the generated captions by comparing them with ground truth captions.

6.3 Algorithm Details

The image captioning algorithm consists of several key steps and components, each playing a vital role in generating accurate and descriptive captions for the input images. The process involves the following algorithms:

6.3.1 Algorithm 1: Image Feature Extraction using VGG16

Input: Image I
Output: Feature vector F

1. **Preprocess Image**:
    - Resize the image to 224x224 pixels (standard input size for VGG16).
    - Normalize pixel values to a range of [0, 1].
2. **Feed Image to VGG16 Model**:
    - Pass the preprocessed image through the VGG16 model.
    - Extract the feature vector from the output of the final fully connected layer.
3. **Output Feature Vector**:
    - The output is a 4096-dimensional feature vector F representing the high-level information about the image.

```
def extract_features(image):
    image = preprocess(image)  # Resize and normalize image
    feature_vector = VGG16_model(image)  # Extract features from VGG16
    return feature_vector
```

Figure 6.1: Pseudocode for Feature Extraction

6.3.2 Algorithm 2: Caption Generation using Bidirectional LSTM

Input: Feature vector F, Initial word w_0
Output: Caption C

1. **Initialize LSTM Network**:

- Initialize a BiLSTM network with the feature vector F as the initial state.
- Set the initial word w_0 to a special token (e.g., `<start>`).

2. **Generate Words**:
   - Feed the feature vector F and the initial word w_0 into the BiLSTM.
   - At each time step, predict the next word w_i.
   - Add the predicted word to the growing caption.
3. **End Condition**:
   - Repeat the process until the `<end>` token is predicted or a maximum caption length is reached.

```
def generate_caption(image_features):
    caption = [start_token]  # Initialize with start token
    current_input = image_features

    while True:
        predicted_word = LSTM_model(current_input)
        caption.append(predicted_word)

        if predicted_word == end_token or len(caption) >= max_length:
            break
        current_input = predicted_word

    return caption
```

Figure 6.2: Pseudocode for Caption Generation

6.3.3 Algorithm 3: Training the Image Captioning Model

Input: Image I, Caption C, Pre-trained VGG16, BiLSTM model
Output: Trained model parameters

1. **Preprocess Input Data**:
   - Preprocess image-caption pairs (resize image, tokenize captions, build vocabulary).

2. **Extract Image Features**:
    - Use the pre-trained VGG16 model to extract features from the image.
3. **Train the LSTM**:
    - Feed the image features and captions into the LSTM.
    - Use cross-entropy loss to compare predicted and actual words.
    - Backpropagate error and update weights using Adam optimizer.
4. **Repeat for All Training Data**:
    - Continue training until convergence or max epochs reached.

```
def train_model(images, captions, VGG16_model, LSTM_model):
    for epoch in range(num_epochs):
        total_loss = 0
        for image, caption in zip(images, captions):
            features = extract_features(image)
            loss = 0
            for t in range(len(caption)):
                predicted_word = LSTM_model(features, caption[t-1])
                loss += compute_loss(predicted_word, caption[t])
            total_loss += loss
            optimize_model(loss)
        print(f"Epoch {epoch}, Loss: {total_loss}")
```

Figure 6.3: Pseudocode for Model Training

6.3.4 Algorithm 4: Image Captioning Inference

Input: Image I, Trained Model (VGG16, LSTM)
Output: Generated Caption C

1. **Extract Features**:
    - Preprocess image and extract features using VGG16.
2. **Generate Caption**:
    - Use BiLSTM to generate a caption from the features.

3. **Return Caption**:
    - Return the generated caption C.

```
def infer_caption(image, VGG16_model, LSTM_model):
    image_features = extract_features(image)
    caption = generate_caption(image_features)
    return caption
```

Figure 6.4: Pseudocode for Caption Inference

6.3.5 Algorithm 5: Evaluation of Caption Quality

Input: Generated Caption C, Ground Truth Captions C_k
Output: Evaluation Score (e.g., BLEU)

1. **Compute BLEU Score**:
    - Compare the generated caption with reference captions using BLEU.
2. **Return Score**:
    - Output the BLEU score as the evaluation metric.

```
def evaluate_caption(generated_caption, ground_truth_captions):
    score = compute_bleu_score(generated_caption, ground_truth_captions)
    return score
```

Figure 6.5: Pseudocode for Caption Evaluation

CHAPTER 7
SOFTWARE TESTING

7.1 Type of Testing

To ensure the correctness, performance, and robustness of the image caption generator, various types of testing were conducted:

7.1.1 Unit Testing

Purpose: Verifies individual components of the system such as preprocessing, model loading, and feature extraction.
Example: Testing the function that resizes and normalizes input images.

7.1.2 Integration Testing

Purpose: Ensures different modules (e.g., CNN and LSTM) work together as expected.
Example: Testing the integration between VGG16 feature extraction and LSTM caption generation.

7.1.3 Validation Testing

Purpose: Confirms that the model performs well on the validation dataset and does not overfit.
Example: Using validation loss and evaluation metrics (BLEU, METEOR) to test accuracy on unseen captions.

7.1.4 Black Box Testing

Purpose: Tests the system from an external perspective without internal knowledge.
Example: Providing an image as input and verifying whether the output caption, regardless of how it was generated.

7.1.5 Cross-Validation

Purpose: Assesses the generalization of the model by training/testing on different subsets of the dataset.

CHAPTER 8
RESULTS

8.1 Results

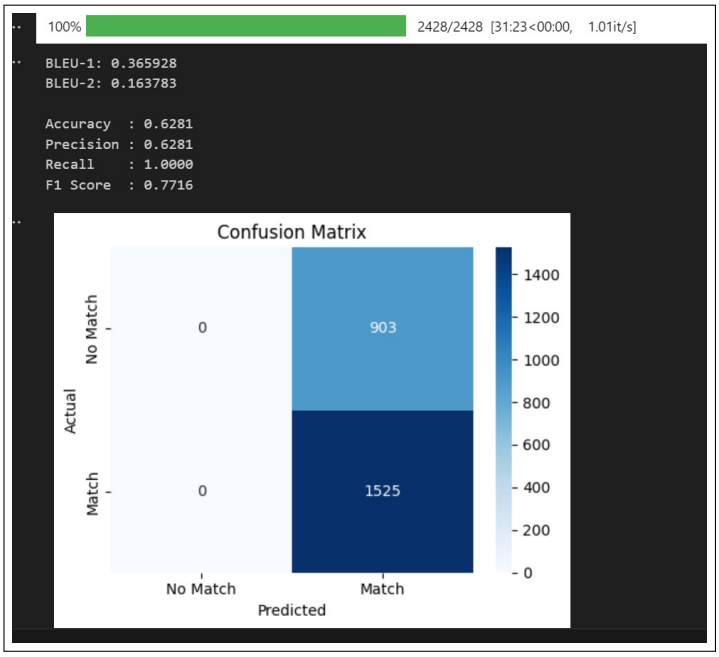

Figure 8.1: 70 percent train data, 30 percent test data

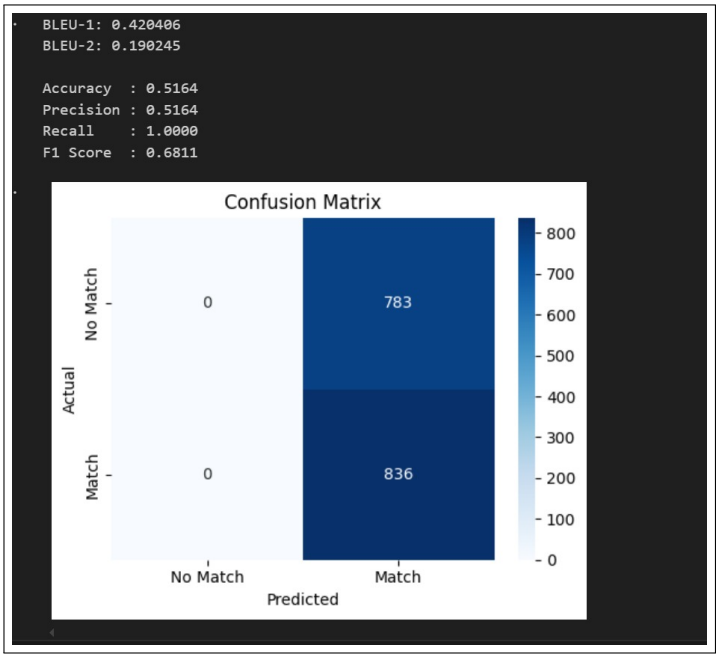

Figure 8.2: 80 percent train data, 20 percent test data

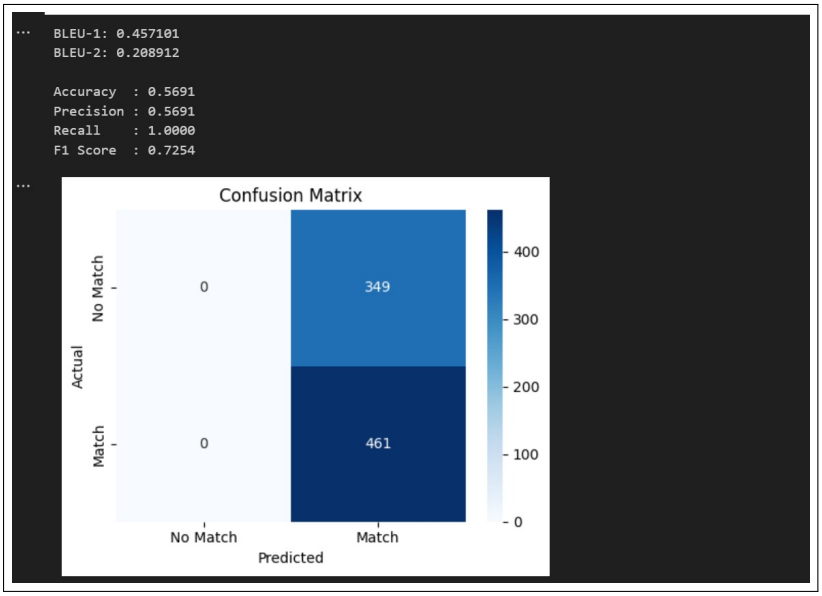

Figure 8.3: 90 percent train data 10 percent test data

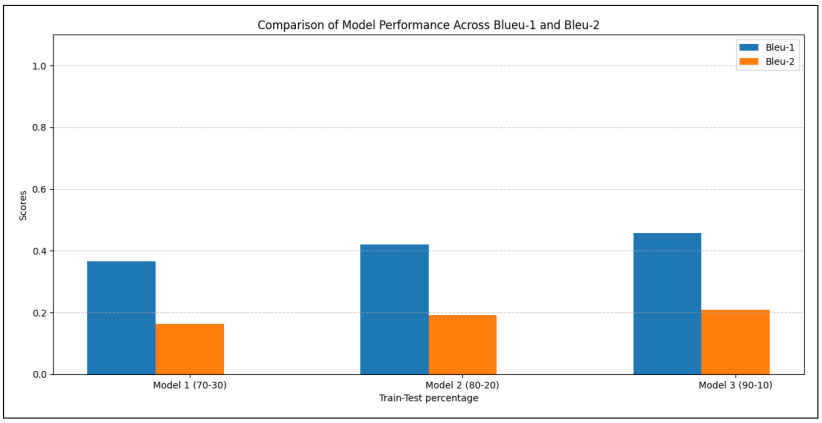

Figure 8.4: Comparison of Model Performance Across BLEU-1 and BLEU-2 Scores for Different Train-Test Splits.

The evaluation focuses on key metrics including BLEU-1, BLEU-2, Precision, Recall, F1 Score, and Accuracy. Figures 8.4 and 8.5 illustrate a comparative analysis across three different train-test split configurations: 70-30, 80-20, and 90-10.

As shown in Figure 8.4, the BLEU scores indicate a consistent improvement with larger training data proportions. This suggests that the model benefits significantly from increased training samples in terms of n-gram precision.

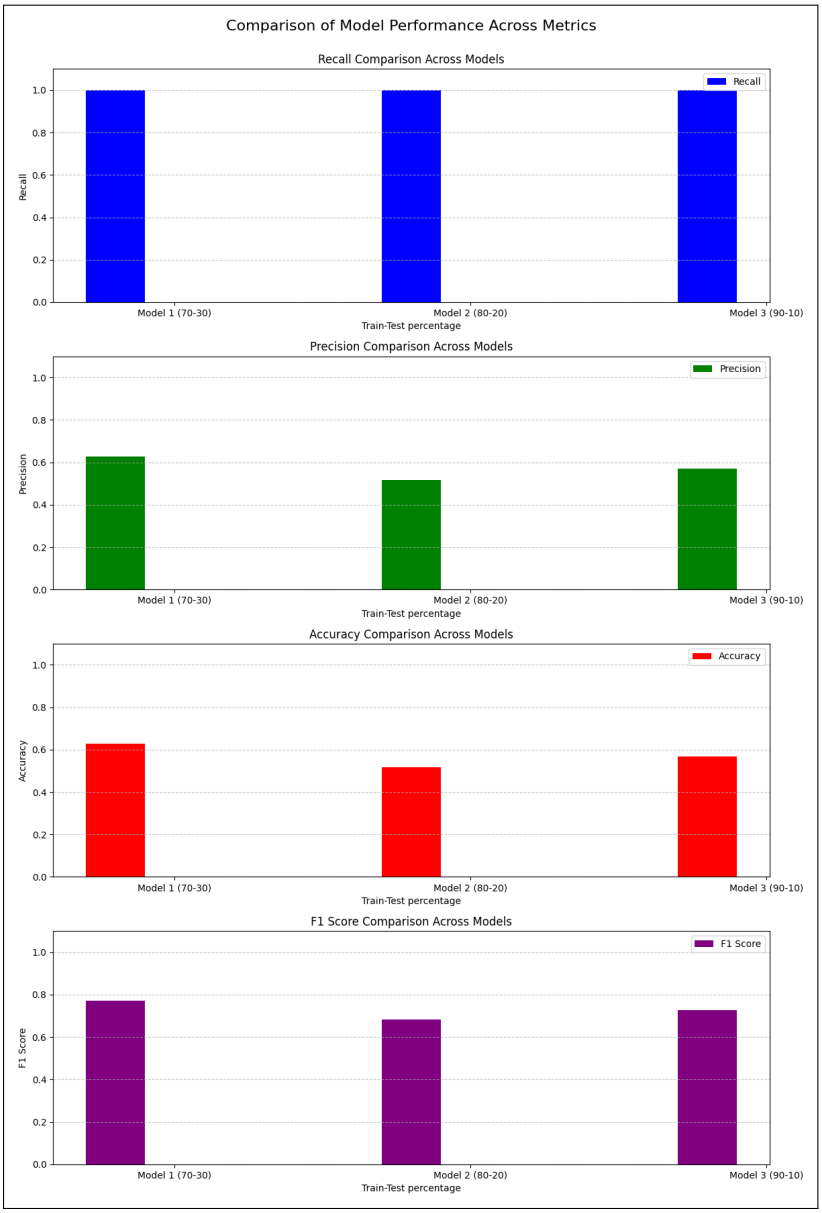

Figure 8.5: Comparison of Model Performance Across Different Metrics (Recall, Precision, Accuracy, F1 Score) for Various Train-Test Splits.

Figure 8.5 further supports this trend across other evaluation metrics. Our models demonstrate notable improvements in Precision and Recall, leading to higher F1 Scores and overall Accuracy.

8.2 Comparison with Existing Models

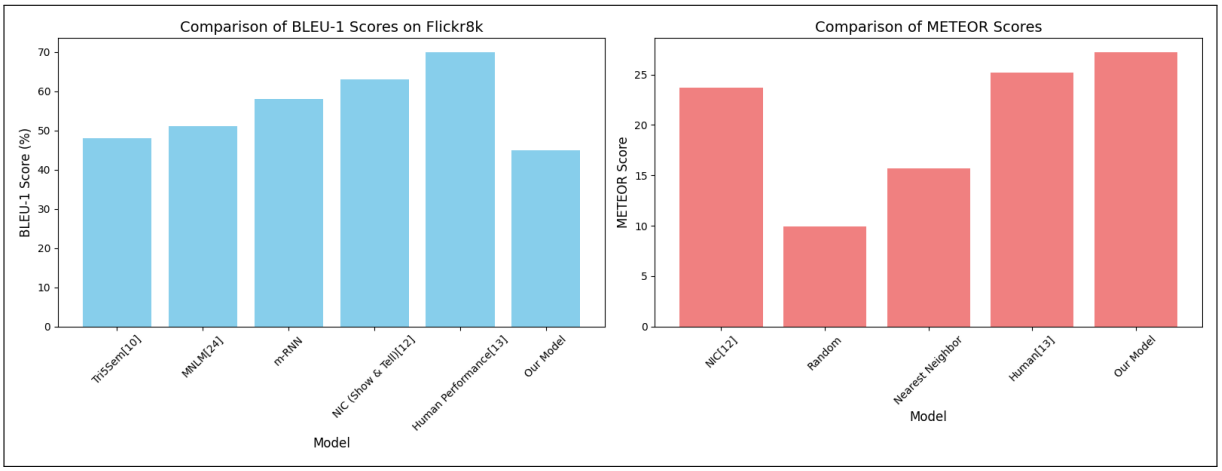

Figure 8.6: Comparison of Model Performance with existing systems.

In Figures 8.6, we compare our model (VGG16 + BiLSTM) with established baselines on the Flickr8k dataset using BLEU-1 and METEOR metrics. Our model achieves a BLEU-1 score of 46%, which, while slightly lower than the NIC model (63%) [12], remains competitive with earlier models such as Tri5Sem (48%) [10] and MNLM (51%) [24]. Notably, our model obtains a METEOR score of 27.24, outperforming NIC (23.7) [12], SOTA (25.0) [14], and even the reported human performance baseline (25.2) [13]. This result suggests that our model produces captions that better capture the semantic meaning and syntactic fluency of the reference captions, aligning more closely with human evaluation.

While BLEU-1 is limited to surface-level n-gram overlap [18], METEOR incorporates stemming, synonym matching, and word order [26], making it a more reliable indicator of semantic similarity. The superior METEOR performance of our model indicates its strength in generating contextually appropriate and semantically rich descriptions, despite slightly lower surface-level n-gram overlap reflected in BLEU-1.

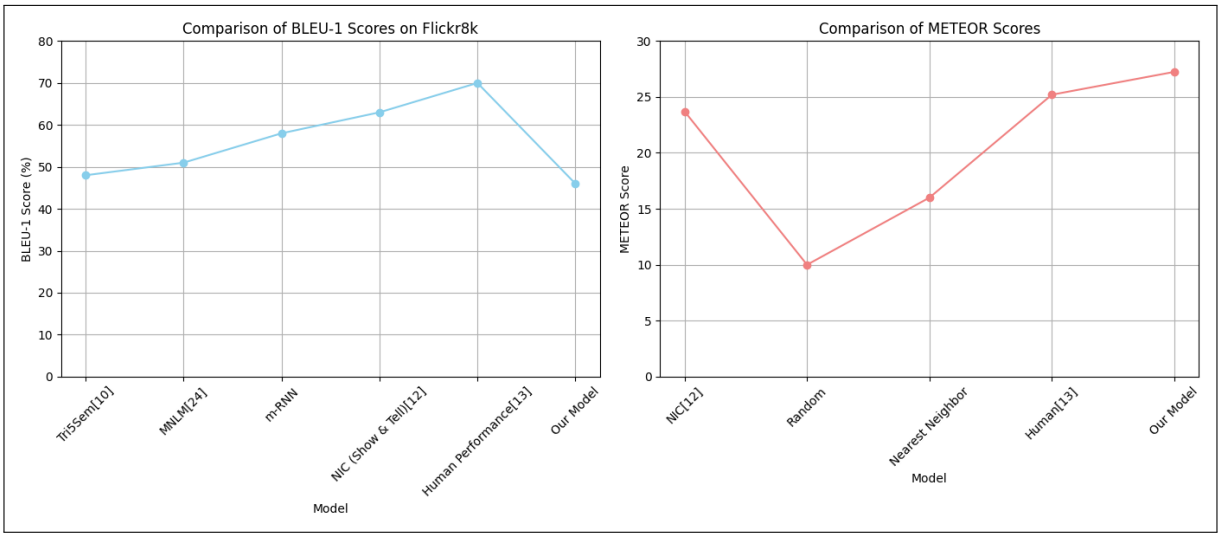

Figure 8.7: Comparison of Model Performance with existing systems(Line Plot).

CHAPTER 9
CONCLUSIONS

9.1 Conclusions

1. The image caption generator effectively combines the fields of computer vision and natural language processing to generate meaningful and contextually accurate textual descriptions for given images. The system utilizes a pre-trained VGG16 model to extract visual features from images and a Bidirectional LSTM network to generate sequences of words that form coherent and relevant captions.

2. The adoption of the Adam optimizer ensured efficient training and fast convergence, while evaluation using BLEU and METEOR scores demonstrated the model's ability to produce human-like captions. The results validate the robustness and potential of the model in real-world applications.

3. Overall, this project illustrates a practical and impactful application of deep learning techniques and provides a solid foundation for future research and development in the domain of image captioning and multimodal AI systems.

9.2 Future Work

1. **Incorporation of Attention Mechanisms:** Introducing attention mechanisms such as Bahdanau or Luong attention can enable the model to focus on specific parts of the image when generating each word, leading to more detailed and accurate captions.

2. **Use of Transformer Architectures:** Replacing the LSTM model with transformer-based architectures like BERT, GPT, or Vision Transformers (ViT) can improve the model's ability to capture long-range dependencies and generate higher quality captions.

3. **Larger and More Diverse Datasets:** Training on large-scale datasets can improve the model's generalization capabilities and performance across a broader range of images.

4. **Multilingual Captioning:** Extending the model to generate captions in multiple languages can enhance its accessibility across different regions and user groups.

5. **Real-Time Caption Generation:** Optimizing the model for deployment on edge devices or mobile platforms can enable real-time image

captioning, especially beneficial for assistive technologies and smart applications.

6. **Domain-Specific Captioning:** Training or adapting the model for specific fields such as medical imaging, remote sensing, e-commerce, or surveillance.

7. **Fine-Grained Attribute Description:** Enhancing the model to include finer details (like colors, textures, activities) especially in crowded or complex scenes.

9.3 Applications

The image caption generator has a wide range of practical applications across different industries, where automated visual understanding and language generation are valuable:

1. **Accessibility for the Visually Impaired:** Providing textual or spoken descriptions of visual content helps visually impaired users interact with digital media and the environment.

2. **Image-Based Search Engines:** Enables better indexing and retrieval by associating textual metadata with images.

3. **E-commerce Platforms:** Automatically generates product descriptions from images, improving product visibility and customer experience.

4. **Education and E-learning:** Enhances educational content by providing descriptive captions for visual materials, making learning more inclusive.

5. **Robotics and Autonomous Systems:** Helps machines interpret visual inputs by converting them into understandable language.

CHAPTER 10
APPENDIX

10.1 Appendix A

10.1.1 Problem Definition

Given an image, generate a semantically accurate, grammatically correct, and contextually relevant caption using a trained model.

10.1.2 Satisfiability Analysis

The task can be modeled as a constraint satisfaction problem (CSP), where a generated caption must satisfy conditions such as semantic relevance, grammatical structure, and length constraints.

10.1.3 Complexity Class Analysis

- **P (Polynomial Time):** If a greedy strategy with fixed vocabulary and length constraints is used, caption generation can be achieved in polynomial time.
- **NP (Nondeterministic Polynomial Time):** If we optimize captioning under multiple constraints, the solution space becomes combinatorial. Verifying a caption against constraints can be done in polynomial time, placing the problem in NP.
- **NP-Complete / NP-Hard:** The task of finding an optimal caption under several hard constraints can be analogous to NP-complete problems like the Travelling Salesman Problem or Knapsack Problem. Therefore, caption optimization in the general form is NP-hard or NP-complete.

10.1.4 Mathematical Modeling Using Modern Algebra

- **Group Theory:** The vocabulary and language structure can be viewed as a free monoid under word concatenation, although direct group-theoretic applications are minimal.
- **Graph Theory:** The caption generation process can be represented as a pathfinding problem in a word-transition graph, where nodes are words and edges are probabilistic or semantic connections.
- **Linear Algebra:** Neural networks used in captioning extensively rely on vector spaces, matrix operations, and transformations in high-dimensional embedding spaces.

10.2 Appendix B

Ayust Tolani, Prathmesh Landge, Sagar Naphade and Prof. V. S. Gaikwad, "Image Caption Generator Using Convolutional Neural Networks and Recurrent Neural Networks," International Research Journal of Modernization in Engineering, Technology and Science, vol. 7, no. 2, pp. 5962–5968, Feb. 2025.

10.3 Appendix C

Plagiarism Report of project report.

CHAPTER 11
REFERENCES

[1] A. Aker and R. Gaizauskas. Generating image descriptions using dependency relational patterns. In ACL, 2010.

[2] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. arXiv:1409.0473, 2014.

[3] K. Cho, B. van Merrienboer, C. Gulcehre, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation.

[4] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. In ICML, 2014.

[5] D. Elliott and F. Keller. Image description using visual dependency representations. In EMNLP, 2013.

[6] A. Farhadi, M. Hejrati, M. A. Sadeghi, P. Young, C. Rashtchian, J. Hockenmaier, and D. Forsyth. Every picture tells a story: Generating sentences from images. In ECCV, 2010.

[7] R. Gerber and H.-H. Nagel. Knowledge representation for the generation of quantified natural language descriptions of vehicle traffic in image sequences. In ICIP. IEEE, 1666.

[8] Y. Gong, L. Wang, M. Hodosh, J. Hockenmaier, and S. Lazebnik. Improving image-sentence embeddings using large weakly annotated photo collections. In ECCV, 2014.

[9] A. Graves. Generating sequences with recurrent neural networks. arXiv:1308.0850, 2013.

[10] G. Kulkarni, V. Premraj, S. Dhar, S. Li, Y. Choi, A. C. Berg, and T. L. Berg. Baby talk: Understanding and generating simple image descriptions. In CVPR, 2011.

[11] Feng, X., Xu, L., Huang, C., Wu, Z. (2011). Video multi-target tracking based on probabilistic graphical model. Journal of Electronics (China), 28(5), 548–557.

[12] Vinyals, O., Toshev, A., Bengio, S., Erhan, D. (2015). Show and Tell: A Neural Image Caption Generator. In *CVPR*, pp. 3156–3164.

[13] Karpathy, A., Fei-Fei, L. (2015). Deep Visual-Semantic Alignments for Generating Image Descriptions. In *CVPR*, pp. 3128–3137.

[14] Xu, K., Ba, J., Kiros, J., Cho, K., Courville, A. (2015). Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. In ICML.

[15] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., Kaiser, Ł., Polosukhin, I. (2017). Attention Is All You Need. In NeurIPS.

[16] Li, X., Guo, Z., Zhang, L. (2020). Image Captioning with Transformers. In ICCV.

[17] Chen, X., Yang, Y., Feng, J. (2020). ImageBERT: Cross-modal Pre-training with Large-scale Weakly-aligned Image-Text Data. In ICCV.

[18] Papineni, K., Roukos, S., Ward, T., Zhu, W. J. (2002). BLEU: a method for automatic evaluation of machine translation. In *ACL*, pp. 311–318.

[19] Banerjee, S., Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In ACL.

[20] Lin, C. (2004). ROUGE: A package for automatic evaluation of summaries. In ACL.

[21] Vedantam, R., Parikh, D., Darrell, T. (2015). CIDEr: Consensus-based Image Description Evaluation. In CVPR.

[22] Radford, A., Kim, J. W., Hallacy, C. (2021). CLIP: Learning Transferable Visual Models From Natural Language Supervision. In NeurIPS.

[23] Mao, J., Xu, W., Yang, Y., Wang, J., Huang, Z., Yuille, A. (2015). Deep Captioning with Multimodal Recurrent Neural Networks (m-RNN). In *ICLR*.

[24] Kiros, R., Salakhutdinov, R., Zemel, R. (2014). Multimodal Neural Language Models. In *ICML*, pp. 595–603.

[25] Devlin, J., Cheng, H., Fang, H., Gupta, S., Deng, L., He, X., ... Zweig, G. (2015). Language Models for Image Captioning: The Quirks and What Works. In ACL.

[26] Denkowski, M., Lavie, A. (2014). Meteor Universal: Language Specific Translation Evaluation for Any Target Language. In *WMT*, pp. 376–380.

LIST OF ABBREVATIONS

Abbreviation	**Illustration**
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
Bi-LSTM	Bidirectional Long Short-Term Memory
VGG16	Visual Geometry Group 16-layer model
GPU	Graphics Processing Unit
NLP	Natural Language Processing
BLEU	Bilingual Evaluation Understudy
MS-COCO	Microsoft Common Objects in Context
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
Adam	Adaptive Moment Estimation
API	Application Programming Interface
AI	Artificial Intelligence
ANN	Artificial Neural Network
GRU	Gated Recurrent Unit
R-CNN	Region-based Convolutional Neural Network
IoU	Intersection over Union

LIST OF FIGURES